

Here is the expanded 12-question curriculum. To get this algorithm into your permanent muscle memory, we are adding problems that take the core mechanic—**swapping elements to their mathematically correct indices**—and applying it to strings, 2D matrices, and disjoint cycles. Work through these in order. By the time you finish Phase 4, your brain will automatically recognize when to use array indices as a built-in hash map.

Phase 1: Core Fundamentals (Missing Numbers)

Focus purely on the $\text{array}[i] = i - 1$ swap logic and tracking what is left behind.

#	LeetCode Problem	Key Concept to Learn
1	268. Missing Number (Easy)	Range 0 to N: Modifying the base pattern to handle a 0-indexed range and safely ignoring the out-of-bounds N.
2	448. Find All Numbers Disappeared in an Array (Easy)	The Second Pass: Running the cyclic sort, then iterating a second time to find all indices where $\text{array}[i] \neq i + 1$.
3	1528. Shuffle String (Easy)	Target Arrays: A perfect conceptual bridge. Instead of $\text{array}[i] = i - 1$, you use an explicit indices array to dictate where characters should be cyclically swapped.
4	1539. Kth Missing Positive Number (Easy)	Tracking Offsets: Often solved with binary search, but excellent for practicing how to count "gaps" between expected and actual values.

Phase 2: Duplicate Detection

Duplicates are the exact inverse of missing numbers. If multiple numbers try to claim the same "correct" index, you have found your duplicate.

#	LeetCode Problem	Key Concept to Learn
5	287. Find the Duplicate Number (Medium)	Collision Detection: When you try to swap an element to its correct index, but the target index <i>already</i> holds that same number, you've trapped the duplicate.
6	442. Find All Duplicates in an Array (Medium)	Multiple Collisions: Applying the Phase 1 second-pass technique to duplicates. Anything sitting at the wrong index at the end is a duplicate.
7	645. Set Mismatch (Easy)	The Hybrid: Finding both the missing number and the duplicate number simultaneously. This proves you've mastered Phases 1 and 2.

Phase 3: The "Cyclic Replacement" Variations

These problems don't sort 1 to N, but they use the exact same underlying philosophy: tracking cycles and swapping elements to calculated target indices in $O(1)$ space.

#	LeetCode Problem	Key Concept to Learn
8	189. Rotate Array (Medium)	Mathematical Cycles: Instead of moving x to $x-1$, you move x to $(i + k) \% n$. You will learn how to detect and break out of infinite swap loops.
9	75. Sort Colors (Medium)	Zone Swapping (Dutch National Flag): Instead of distinct indices, elements

		belong to "zones" (0s left, 2s right). You use index-pointers to cyclically swap them into place.
10	<u>48. Rotate Image</u> (Medium)	2D Cyclic Swaps: Applying cyclic replacement to a matrix. You swap 4 elements in a cycle A -> B -> C -> D -> A moving layer by layer.

Phase 4: Ultimate Mastery

These are the heavy hitters. If you can solve these, your understanding of cyclic sort and index-mapping is bulletproof.

#	LeetCode Problem	Key Concept to Learn
11	<u>41. First Missing Positive</u> (Hard)	Ignoring Garbage Data: The defining Cyclic Sort problem. You must adapt the algorithm to ignore negatives, zeros, and numbers larger than N while sorting the rest in place.
12	<u>765. Couples Holding Hands</u> (Hard)	Disjoint Cycles: The most advanced application of swapping to correct positions. You must resolve cycles of misplaced pairs using greedy cyclic swaps.